



Launch a Kubernetes Cluster



Mohammed Dawoud Mota

The screenshot shows the AWS Management Console interface for a Kubernetes cluster. The 'Compute' tab is selected, and the 'Nodes' page is displayed. The table lists 3 nodes, all of which are in a 'Ready' status. The nodes are managed by the 'nextwork-nodegroup' and are of the 't3.micro' instance type. The table includes columns for Node name, Instance type, Compute, Managed by, Created, Status, CPU usage, and Memory.

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory
ip-192-168-17-16.ec2.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%	66%
ip-192-168-39-244.ec2.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	22%	59%
ip-192-168-41-135.ec2.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%	66%



Introducing Today's Project!

In this project, I am here to launch my first Kubernetes cluster using Amazon EKS so I can understand how Kubernetes works in a real cloud environment. This includes learning how to deploy an EC2 instance, install eksctl, create an EKS cluster, and see how AWS services like CloudFormation, IAM, and EC2 work together behind the scenes. I am doing this because Kubernetes is the standard tool for running and scaling containerized applications, and gaining hands-on experience with EKS helps me build the foundational skills I need for modern cloud and DevOps work.



What is Kubernetes?

Kubernetes is a container orchestration platform that keeps containers running smoothly across all the servers in a cluster. It automatically handles tasks like scaling containers up or down, restarting them if they fail, balancing traffic, and making sure everything stays healthy without constant manual work.

Companies and developers use Kubernetes because it removes the need to manage containers one by one. It makes large applications easier to operate, more reliable, and able to handle changes in traffic without downtime. This lets teams focus on building features instead of managing infrastructure.

I used `eksctl` to create my Kubernetes cluster on Amazon EKS. After installing it on my EC2 instance, I ran the `eksctl create cluster` command, which automatically set up the entire cluster for me. `eksctl` handled creating the control plane, the node group, and all the networking resources behind the scenes through CloudFormation. Using `eksctl` made the process much easier because it automated tasks that would have required many manual steps with the AWS CLI.

I ran into errors with `eksctl` because my EC2 instance did not have the permissions it needed to create an EKS cluster. When I first tried to run the command, the instance had no IAM role attached to it, so AWS treated it like a computer that was not signed in to my account. Without the right permissions, `eksctl` could not create the cluster or access the services it needed. The error was expected and helped show why attaching an IAM role with the proper access is required before the cluster can be created successfully.



```

#
~\ \ #####\
~\ \ #####\
~\ \ #####|
~\ \ ##/\
~\ \ #/\
~\ \ v~' ->
~\ \ /-
~\ \ .-
~\ \ m/ '-

[ec2-user@ip-172-31-16-65 ~]$ eksctl create cluster \
--name nextwork-eks-cluster \
--nodegroup-name nextwork-nodegroup \
--node-type t3.micro \
--nodes 3 \
--nodes-min 1 \
--nodes-max 3 \
--version 1.33
-bash: eksctl: command not found
[ec2-user@ip-172-31-16-65 ~]$

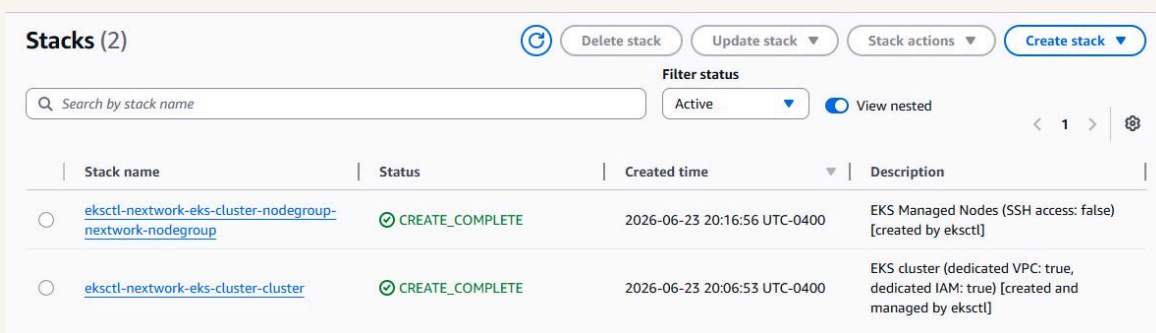
```



eksctl and CloudFormation

CloudFormation helped create my EKS cluster because eksctl uses it behind the scenes to build all the resources the cluster needs. When I ran the eksctl create cluster command, it automatically generated CloudFormation stacks that created the VPC, subnets, route tables, security groups, and the node group. CloudFormation handled each step of the setup and showed me every event taking place, which made it easy to track the progress and understand how the cluster was being assembled.

There was a second CloudFormation stack because eksctl creates the EKS cluster and the node group separately. The first stack builds the core EKS cluster, including the control plane and all the networking resources. The second stack is dedicated to the node group, which is the group of EC2 instances that will run the containers. Splitting them into two stacks makes it easier to manage each part independently and helps with troubleshooting if something goes wrong.



The screenshot shows the AWS CloudFormation console interface. At the top, there are buttons for 'Delete stack', 'Update stack', 'Stack actions', and 'Create stack'. Below these is a search bar labeled 'Search by stack name' and a 'Filter status' dropdown set to 'Active'. A 'View nested' toggle is also visible. The main content is a table with two stacks, both in a 'CREATE_COMPLETE' status.

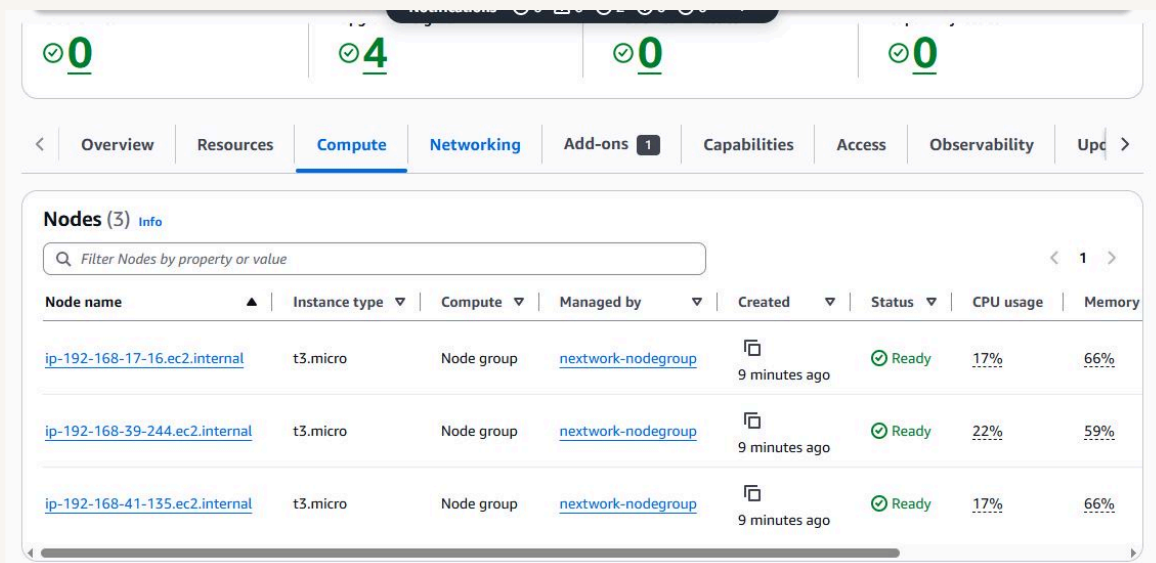
	Stack name	Status	Created time	Description
☐	eksctl-nextwork-eks-cluster-nodegroup-nextwork-nodegroup	CREATE_COMPLETE	2026-06-23 20:16:56 UTC-0400	EKS Managed Nodes (SSH access: false) [created by eksctl]
☐	eksctl-nextwork-eks-cluster-cluster	CREATE_COMPLETE	2026-06-23 20:06:53 UTC-0400	EKS cluster (dedicated VPC: true, dedicated IAM: true) [created and managed by eksctl]



The EKS console

I set up an IAM access entry so my IAM user could interact with the EKS cluster through the AWS Management Console. Even though the cluster was created successfully, my user did not automatically have permission to view or manage the nodes inside it. The access entry connects my IAM identity to Kubernetes RBAC, which allows the cluster to recognize me and grant the right level of access. By attaching the AmazonEKSClusterAdminPolicy, I made sure I could fully manage the cluster and see all of its resources.

It took around 30 minutes to create my cluster. Since I'll create this cluster again in the next part of the project, maybe I can speed up the deployment process.



The screenshot shows the AWS EKS console interface. At the top, there are four summary cards with green checkmarks and numbers: 0, 4, 0, and 0. Below these is a navigation bar with tabs: Overview, Resources, Compute (selected), Networking, Add-ons (1), Capabilities, Access, Observability, and Upgrades. The main content area is titled 'Nodes (3)' and includes a search bar. Below the search bar is a table with columns: Node name, Instance type, Compute, Managed by, Created, Status, CPU usage, and Memory. The table contains three rows of node information, all with a 'Ready' status.

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory
ip-192-168-17-16.ec2.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%	66%
ip-192-168-39-244.ec2.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	22%	59%
ip-192-168-41-135.ec2.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%	66%



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

